SMITH ENGINEERING

Queen's University

Department of Electrical and Computer Engineering



ELEC 490/498 Project Blueprint

[Mining Software Repositories GitSkills Dataset Challenge]

Fall 2026 / Winter 2027

Submitted by Group [34]

Group member	NetID	Student number
[Noah Chung]	[20ntc2]	[Student number]
[Faiz Farook]	[21ff8]	[Student number]
[James Kilpatrick]	[21jpk5]	[20357509]
[Naman Nagia]	[21nn48]	[20357592]

Faculty supervisor(s): [Mariam Guizani]

Supervisor email: [mariam.guizani@queensu.ca]

Submission date: [07 October 2026]

Version: [1.0]

Executive Summary

AI coding agents increasingly follow *agent skills*: `SKILL.md` files of natural-language instructions that an agent loads when it judges that a task matches the skill's description. Within nine months of the format's release, public GitHub repositories held 3.8 million skill files [1]. Skills are written mainly in prose, no compiler checks them, and they spread by being copied between repositories. Software engineering has long used metrics such as size, churn, age, clone coverage, and readability to judge source code, but it is not known whether these metrics carry over to skills. Using the GitSkills dataset from the Mining Software Repositories (MSR) 2027 Mining Challenge [2], this project asks which of these metrics have meaningful equivalents for skills and how their distributions compare with those of source code. The end product is a reproducible analysis pipeline, documented metric definitions, visualizations, and a report interpreting the findings for researchers and developers who write and reuse skills.

The pipeline loads the GitSkills tables, filters out files that are not genuine skills, runs a separate calculation script for each metric, and joins the results into a single metrics file. Each metric is adapted to skills. Size is counted in tokens, separately for the description, the body, and bundled files. Churn is split by type, because skills are more often copied and modified than edited in place. Age is measured from the first commit. Clone coverage is computed from identical content hashes. Readability scores each part of a skill with an instrument suited to it and is validated against human ratings. Performance targets include a full-dataset run within 24 h using at most 16 GB of memory, identical results on re-running, and at least 0.90 precision for cross-copy lineage links. The main milestones are a prepared and checked dataset (M2, 13 November 2026), all supported metrics calculated with initial plots (M6, 4 December 2026), and a validated, reproducible final pipeline (M12, 12 March 2027).

Every metric is first prototyped on the official 13,000-skill GitSkills sample before being scaled to the full dataset. Pilots have already run: they show that about two-thirds of skills are never edited after being added, and that skill bodies read at a median Flesch–Kincaid grade of 8.6. Testing combines unit tests against hand-calculated examples, integration tests across pipeline stages, reproducibility checks, and manual review of sample outputs. The dataset is openly licensed, all tools are open source, and computation fits on a single workstation, so the estimated total cost is **[$0]** CAD. The team is ready to begin. Further learning is needed in large-scale data handling, text-similarity and readability measurement, and non-parametric statistics.

Contents

Executive Summary i

Acronyms vi

1 Introduction 1

1.1 MSR Challenge and Project Motivation . . . 1

1.2 Project Objective and Research Direction . . . 1

1.3 Scope and Deliverables . . . 2

2 Functional Description 3

2.1 System overview . . . 3

2.2 Interface and performance specifications . . . 6

2.3 Risks and safety hazards . . . 8

3 Design and Production Approach 9

3.1 Metric Development and Analysis . . . 9

3.2 Milestones and Division of Labor . . . 11

3.3 Further Exploration and Contingencies . . . 12

4 Testing and Evaluation 13

4.1 Metric Verification and Validation . . . 13

4.2 Pipeline Verification . . . 14

4.3 Evaluation and Acceptance . . . 15

5 Resource Requirements 16

5.1 Scheduling access to resources . . . 16

5.2 Materials and other resources . . . 16

6 Conclusion 17

7 References 17

A Supporting Schedule and Technical Material: N/A 19

B Readability Pilot **20**

List of Figures

1 Data flow from the GitSkills tables through the Python pipeline to the final metrics file. 2

2 Size and Churn modules, showing the fields read from GitSkills, the shared pre-processing, and the outputs of each calculation. 4

3 Readability pilot on 8,314 English skills from the 13,000-skill GitSkills sample. (a) Flesch–Kincaid grade of the body prose, with the Markdown converted to sentences and as raw text. (b) Body grade under four formulas, and Flesch–Kincaid grade of the front-matter description. Boxes show the interquartile range, and whiskers extend to 1.5 times the IQR. 20

List of Tables

1 Preliminary system specifications. 6
2 Planned milestones and lead responsibilities 11
3 Contingencies and affected milestones 12
4 Planned metric verification and validation 13
5 Planned pipeline verification 14
6 Resource access by milestone. 16
7 Preliminary cost estimate (CAD). 16

Acronyms

MSR Mining Software Repositories

1. Introduction

1.1 MSR Challenge and Project Motivation

The Mining Software Repositories (MSR) Mining Challenge invites researchers to investigate development practices using shared software repository datasets. The 2027 challenge provides GitSkills, a dataset containing approximately 3.8 million occurrences of agent skill files [2]. An agent skill combines natural-language instructions in a `SKILL.md` file with optional supporting scripts and references. Skills can spread through copying between repositories, and no compiler or type checker verifies whether an agent selects the appropriate skill [2]. Understanding skill duplication, revision activity, and instruction complexity could provide insight into how these artifacts are reused and maintained. However, applying standard software metrics to skills requires examining what those measurements capture and whether their interpretation remains meaningful.

Our team will use GitSkills to study how agent instructions are written, reused, and maintained in software environments. These findings could help developers and researchers identify patterns in the development and evolution of this emerging software artifact.

1.2 Project Objective and Research Direction

The project builds on MSR's question of whether established software metrics such as size, churn, age, clone coverage, and readability have meaningful equivalents for agent skills [2]. Prior work has evaluated code readability against human judgements, providing a precedent for checking whether calculated metrics reflect their intended interpretation [3]. Applying such measurements to agent skills requires checking their suitability for natural-language instructions and any accompanying code or supporting material. The team will first define each metric, verify its calculation, and evaluate its interpretation for agent skills. Supported measurements will then be used to examine distributions and relationships, with inconclusive findings and measurement limitations reported explicitly. Potential analyses include how instruction complexity relates to revision activity and how duplicated content is distributed across skill collections. The engineering objective is to develop a reproducible pipeline that processes and validates data, calculates metrics, and produces visualizations supporting conclusions about skill development, reuse, and maintenance.

1.3 Scope and Deliverables

The initial scope is to explore GitSkills and assess which metrics support meaningful analysis of agent instructions. Specific research questions will be refined using preliminary results, while the core work remains metric definition, validation, and reproducible analysis. SpecMine is excluded to focus efforts on agent skills. Non-English skills will be counted and included in applicable metrics, but excluded from English-language readability scoring. The study will characterize repository artifacts and their relationships; executing skills to evaluate agent performance and establishing causal effects on skill quality are outside scope.

Deliverables include documented metric definitions, reproducible analysis code, visualizations, and a report interpreting the findings. Analyses requiring unavailable history or unsupported measurements will be narrowed or omitted, with exclusions and their implications documented. The team intends to submit to the MSR 2027 Mining Challenge, subject to having sufficiently developed and validated findings for the December 18, 2026 abstract and December 23, 2026 paper deadlines [2]. Otherwise, work will continue according to the capstone schedule without compromising rigor to meet the conference deadline.

A researcher will run the pipeline on a workstation using a fixed GitSkills snapshot and recorded configuration. External inputs comprise the dataset and selected repository histories; the system boundary covers data preparation through metric calculation and output generation. Operation assumes sufficient local storage and access to required histories, with batching used to manage memory limits. Figure 1 provides an overview of this pipeline; Section 2 describes its components and interfaces.

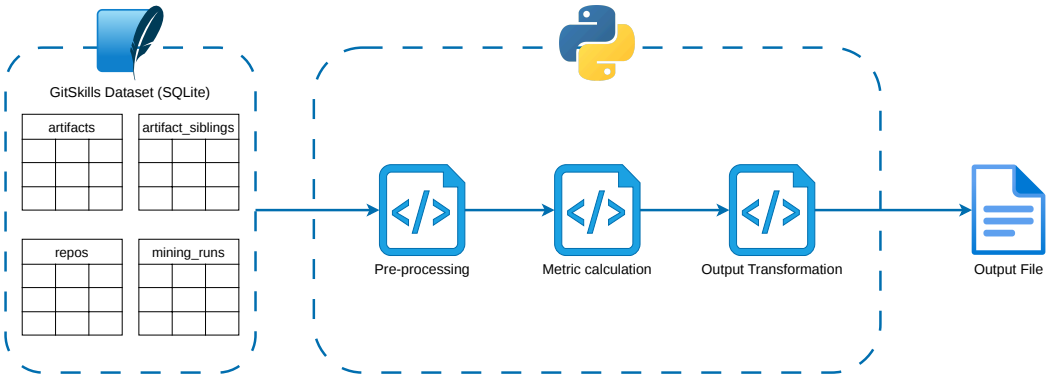


Figure 1: Data flow from the GitSkills tables through the Python pipeline to the final metrics file.

2. Functional Description

2.1 System overview

The pipeline introduced in Figure 1 comprises data preparation, metric calculation, and output transformation. GitSkills is distributed as a single SQLite database, with a Parquet copy of each table, and holds four tables: `artifacts`, `artifact_siblings`, `repos`, and `mining_runs`. The fields we need are queried from these tables into Python, where we rely on common data-analysis libraries: pandas and NumPy for data handling, DuckDB and PyArrow for querying the Parquet files, Anthropic's tokenizer (loaded through the Hugging Face tokenizers library) for token counts, textstat for readability formulas, datasketch for MinHash similarity, SciPy and powerlaw for statistics, and Matplotlib for plots. A single pre-processing script, shared by every metric, first transforms it for the metric scripts and filters out files that are not genuine skills, keeping one row per distinct skill (*dedup_primary* = 1, a *filename* of SKILL.md, valid front matter, not a symlink, and a non-empty body). It also splits each skill's *content* into front matter and body, and the body into prose and code. The cleaned data then passes to a separate calculation script for each metric, whose methods are described in Section 3. Finally, an output transformation script joins the results into a single metrics file used for analysis, plotting, and validation. The only input from outside the dataset is a small sample of cloned GitHub repositories used to validate churn.

Size and Churn Calculation Modules.

Figure 2 shows the Size and Churn modules in one diagram to save space, since both start from the same pre-processing step. The tables on the left list every field the two modules read, all keyed by *file_sha* except bundle files, which join to their skill on *repo_full_name* and *path*. Clone Coverage is faded because Churn reuses its near-clone families instead of building its own.

Size Calculation Module.

The Size module counts the tokens in each of a skill's three tiers: the description, the body (prose and code), and the bundle, which only counts the text files whose content GitSkills stored. Skills with *composition_truncated* = 1 are flagged, since their bundle listing is incomplete. Lines and words are also counted for the body, along with lines and tokens for each bundled script, so skills can be compared directly with ordinary code. The output is one row of size columns per *file_sha*, plus a separate file for the bundled scripts.

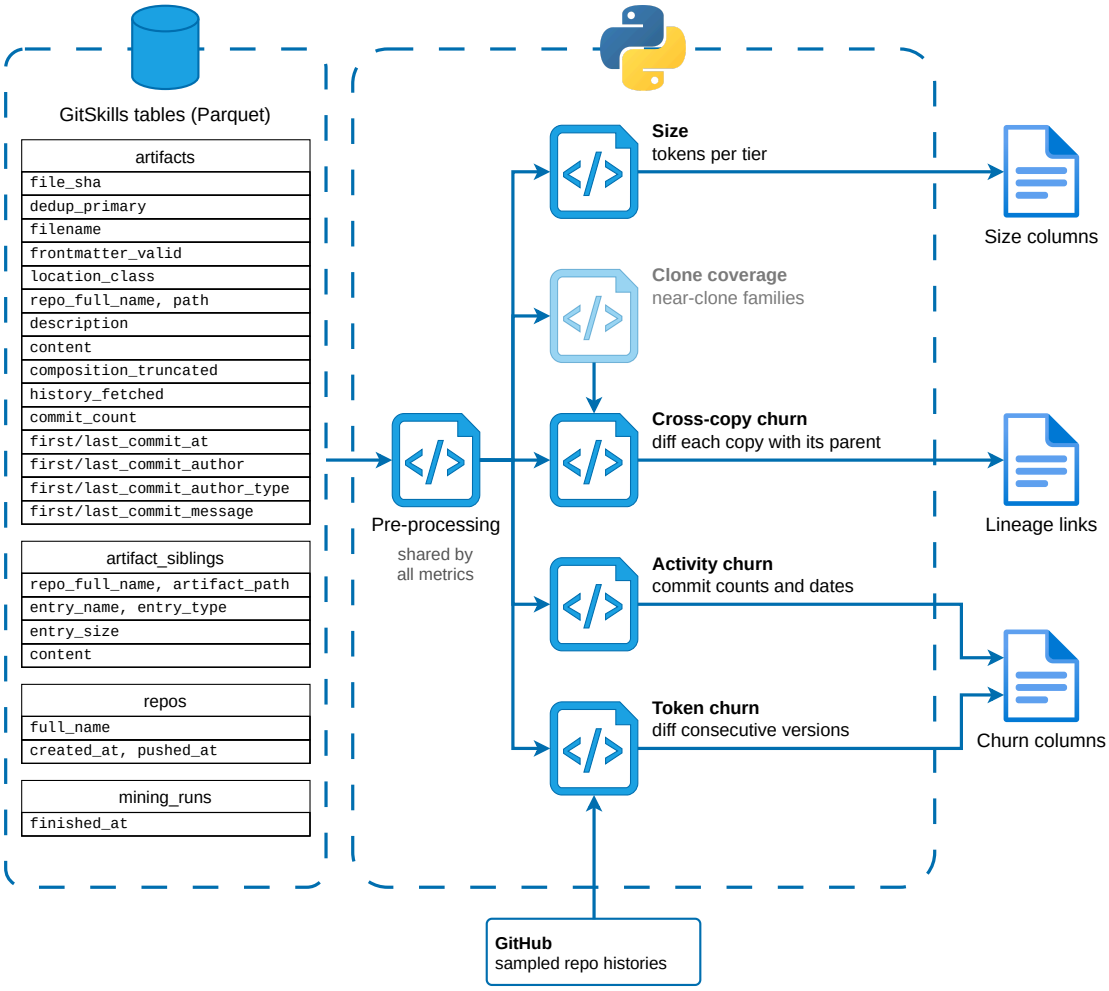


Figure 2: Size and Churn modules, showing the fields read from GitSkills, the shared pre-processing, and the outputs of each calculation.

Churn Calculation Module.

The Churn module splits into three calculations. *Activity churn* uses the commit fields of every skill with history (*history_fetched* = 1) to find how many times it was edited (*commit_count* minus 1), its edits per month, how long it was actively edited, and how long it has sat untouched up to the snapshot date (*finished_at* in *mining_runs*). It also flags first or last commits that were AI co-authored or made by a bot, and whether the author changed between them. *Token churn* follows a sample of edited skills, stratified by *location_class*, through their cloned histories, renames included, and counts the tokens added and removed between each pair of consecutive versions. *Cross-copy churn* orders each near-clone family by *first_commit_at*, or by the repository's *created_at* when a skill has no history, then compares each skill with its most similar earlier member and counts the tokens added and removed. Activity and token churn go into the churn columns, while each

link from a parent to its child goes into a separate lineage file, along with its similarity, its token counts, and which date was used to decide its direction.

Age Calculation Module.

The age module calculates how long the SKILL.md file has been inside the repository. It uses the `first_commit_at` timestamp and a fixed dataset collection date to calculate the age of the skill. The purpose of the fixed dataset collection date is to assure consistency across the dataset. The module will output the calculated age value (displayed in days). This value will be passed to the output stage for analysis and potential visualization. Records with missing or invalid values in the textttfirst_commit_at field will not be imputed but will be flagged instead. The use of the first commit date has been selected to determine age over the latest commit date as the team looks to measure how long the skill has existed rather than how long since the skills final edit.

Clone Coverage Calculation Module.

The Clone Coverage module measures how frequently identical or substantially similar AI agent skills appear within the GitSkills dataset. Exact clones are identified by grouping skill occurrences with matching `file_sha` values. Groups containing multiple occurrences are classified as exact clone groups, and exact clone coverage is calculated as the percentage of eligible skill occurrences belonging to these groups, including the first occurrence.

Near-clones will be identified using the actual skill text stored in the `content` field, regardless of repository or filename. The `dedup_primary` field will be used to select representatives of exact-content groups, while `content_fetched` and `content_sha_ok` will verify text availability and integrity. Skill text will be divided into overlapping three-token sequences (shingles), and MinHash signatures with locality-sensitive hashing (LSH) will identify potentially similar pairs. These pairs will then be evaluated using Jaccard similarity, calculated as the number of shared shingles divided by the total number of unique shingles across both skills.

Pairs with different file hashes and Jaccard similarity of at least 80% will be linked as near-clones. This threshold follows published text deduplication research and requires substantial textual overlap while allowing minor modifications. Qualifying relationships will be combined into connected clone groups, allowing multiple related skills to be grouped together even when not every pair exceeds the threshold. These groups will be mapped back to the original skill occurrences, and expanded clone coverage will be calculated as the percentage of eligible occurrences belonging to groups containing multiple skills.

Exact and expanded clone coverage will be reported separately, with missing or invalid data handled separately.

2.2 Interface and performance specifications

Table 1: Preliminary system specifications.

ID	Requirement	Target	Tolerance
IF-01	All four GitSkills tables load into Python with their documented columns	4 of 4 tables	0 missing columns
IF-02	Scripts pass data keyed by `file_sha` (bundle files by *repo_full_name* and *path*), with no rows lost between stages	100% of rows kept	0 rows
IF-03	Output file holds one row per distinct skill, in Parquet and CSV	1 row per skill	0 duplicates
PF-01	Full-dataset run time on one workstation	$\leq$ 24 h	+12 h
PF-02	Peak memory use, kept low through batching	$\leq$ 16 GB	+4 GB
PF-03	Re-running the pipeline reproduces the same metrics	Identical output	0 differences
PF-04	Precision of cross-copy lineage links on the hand-labelled sample	≥ 0.90	-0.05
PF-05	Spearman correlation between Flesch–Kincaid grade and the mean human rating on the 100-skill readability sample	$\lvert\rho\rvert \geq 0.5$	-0.1
PF-06	Agreement between the two readability raters (Krippendorff's α [4])	≥ 0.80	-0.13
FR-01	For every included skill, the Size module shall report the token count of its description, body, and bundle	3 counts per skill	0 missing*
FR-02	For every included skill with *composition_truncated* = 1, the Size module shall flag its bundle count as unreliable	100% of truncated skills flagged	0 unflagged

ID	Requirement	Target	Tolerance
FR-03	For every included skill, the Size module shall report the non-blank line count and word count of its body	2 counts per skill	0 missing*
FR-04	For every bundled script with stored content, the Size module shall report its non-blank line count and token count	2 counts per script	0 missing*
FR-05	For every included skill with *history_fetched* = 1, the Churn module shall report its edits, edits per month, active span, and dormancy	4 values per skill	0 missing*
FR-06	For every included skill with *history_fetched* = 1, the Churn module shall flag whether its first or last commit was AI co-authored or made by a bot	1 flag per skill	0 missing*
FR-07	For every sampled skill whose repository can be cloned, the Churn module shall report the tokens added and removed between each pair of consecutive versions	Every cloned sampled skill	0 missing*
FR-08	For every near-clone family, the Churn module shall link each member except the earliest to exactly one earlier parent	$n-1$ links per family of n	0 links
IF-04	Clone Coverage passes a *family_id* for every included skill in a near-clone family, keyed by *file_sha*	1 ID per member	0 missing*
IF-05	Lineage file holds one row per link with *parent_sha*, *child_sha*, *similarity*, *tokens_added*, *tokens_removed*, and *direction_basis*	6 fields per link	0 missing*
PF-07	Token, line, and word counts on the 20 skills chosen for T01 match a standalone reference script	Exact match	0 differences

ID	Requirement	Target	Tolerance
PF-08	Tokens added and removed on the version and copy pairs chosen for T02 match git's word diff of the same pairs	Within 2 tokens per pair	0 pairs outside
PF-09	Share of links with both *first_commit_at* values where ordering by repository *created_at* gives the same parent	≥ 0.85	-0.05

*Depending on how clean the data is, a few outlier records may lack the usual fields. These are flagged and reported rather than counted as missing.

2.3 Risks and safety hazards

The project is purely computational, so it carries no physical, health, or safety hazards. The main technical risk is the dataset's size (about 44 GB), which is managed by reading the per-table Parquet files in batches rather than loading everything into memory. Since the data comes from public GitHub repositories with commit authors already anonymized, we will not attempt to identify individuals, and we will respect each repository's license when quoting skill content.

3. Design and Production Approach

3.1 Metric Development and Analysis

The following outlines the proposed methods for measuring size, churn, age, clone coverage, and readability, including the data requirements, calculation procedures, and validation approach for each metric. Each metric follows four rough stages: it is prototyped on the official 13,000-skill GitSkills sample, calibrated, scaled to the full dataset, and validated.

Size. Size is classically measured in lines, but that doesn't translate well to skills. A skill mixes plain-language instructions with code blocks, and a line of prose and a line of code carry very different amounts of information; in our pilot, about 12% of a typical skill's body was code. Instead, we measure size in tokens, the units an agent actually reads text in and the units its context window is limited by, counted with Anthropic's tokenizer. We split that count across the three parts of a skill, since an agent loads each at a different time: the *description*, read for every installed skill to decide which one to use; the *body*, loaded only once a skill is chosen; and the *bundle*, the scripts and reference files stored next to the skill, which the body points to when it needs them. Measuring each part separately shows not just how big a skill is, but where that size costs the agent. Lines and words are still recorded alongside tokens, so skills can be compared directly with the code bundled next to them.

Churn. Classically, churn is the number of lines added plus lines removed between commits, and it shows how much a file changes as it develops and what patterns those changes follow. Since lines don't fit skills well, churn needs adapting too, and the data adds a second complication: GitSkills records how often and when a skill was committed, but not what changed, and about two-thirds of skills in our pilot were never edited after being added. What skills tend to do instead is get copied and then modified for a new project, so we split churn into three types. *Activity churn* uses the recorded commit counts and dates to measure how often, and for how long, a skill is maintained. *Cross-copy churn* takes the near-clone families found by Clone Coverage (MinHash), links each skill to the most similar member that appeared before it, and counts the tokens added and removed along the way. Which copy came first is decided by the first commit date, or by the repository's creation date when a skill has no history, and a sample of the links is checked by hand (PF-04). This is where we expect the most useful results, since it captures how popular templates get adapted for uses like design or security. *Token churn*

is classical churn counted in tokens instead of lines, computed from the full git history of a sample of edited skills, and confirms that the other two measures reflect real change.

Age. Age is usually measured as the amount of time since the file was introduced to the repository. This method of measurement will work well for skills. The GitSkills artifact subset has fields: first_commit_at and last_commit_at. The team will calculate the difference between first_commit_at and the dataset collection date, then convert this difference to days to create the Age metric.

Clone Coverage. Clone coverage measures how much content in a dataset is duplicated. In the GitSkills artifacts subset, there is a field called content_hash. Clones can be identified by checking to see if they have the same content_hash. To calculate the clone coverage percentage, the team will group skills by content_hash and count the number of cloned skill occurrences. The number of cloned skill occurrences will then be divided by the total number of skill occurrences then multiplied by 100 to get the clone coverage percentage.

$$\text{Clone Coverage} = \frac{\text{Number of cloned skill occurrences}}{\text{Total number of skill occurrences}} \times 100.$$

Readability. Readability is classically measured in two separate ways. Formulas such as the Flesch–Kincaid grade level score English prose by sentence and word length [5], [6], while code readability models score code from features such as line count and token entropy [3], [7]. Neither fits a skill on its own, so we split each skill into the same three parts used for size and score each part with the instrument designed for it. The body's prose and the description are scored with the Flesch–Kincaid grade, with three related formulas as a cross-check. The body's code blocks are scored with Posnett's model, which is also applied, in equal-length snippets, to the scripts bundled next to the skill, so skill code and ordinary code are compared on the same scale. Before scoring, the Markdown has to be turned into real sentences. Otherwise each bullet list counts as one long sentence, and in our pilot on the 13,000-skill sample this raised the median skill's grade by 3.4 levels (Appendix B). The formulas only work for English, so non-English skills (about 21% of the pilot) are counted but not scored. In the pilot, the median skill body read at grade 8.6, while descriptions read at grade 12.7. To validate, two team members will independently rate 100 skills, stratified by grade, on a five-point scale. The formula is accepted if the raters agree with each other (PF-06) and its scores track their average rating (PF-05). If it fails, readability will be reported only as a measure of text complexity, not of quality.

3.2 Milestones and Division of Labor

Development will follow an iterative approach: metric implementations will be tested on sample data and refined before integration and larger-scale analysis. Progress and outstanding issues will be reviewed during Friday team meetings. Each milestone lead will coordinate completion and supporting test evidence, with implementation and review shared across the team.

Table 2: Planned milestones and lead responsibilities

ID	Milestone	Target date	Lead
M1	Metric definitions reviewed	Nov. 6, 2026	Noah
M2	Dataset preparation completed	Nov. 13, 2026	Naman
M3	Initial metric calculations verified	Nov. 13, 2026	James
M4	Sample pipeline integrated	Nov. 20, 2026	Faiz
M5	Scalability assessed	Nov. 27, 2026	Naman
M6	Initial analysis produced	Dec. 4, 2026	Faiz
M7	MSR submission-readiness review	Dec. 11, 2026	James
M8	MSR abstract submitted (conditional)	Dec. 18, 2026	Faiz
M9	MSR paper submitted (conditional)	Dec. 23, 2026	Noah
M10	Results reviewed and focused research questions documented	Jan. 29, 2027	Noah
M11	Follow-up investigations evaluated	Feb. 26, 2027	Naman
M12	Final pipeline validated and results reproduced	Mar. 12, 2027	James

M2 and M3 are in progress. Dataset preparation has supported preliminary assessments, but final preprocessing checks and initial calculation verification remain ongoing.

M1 establishes the definitions used for M3. Completion of M2 and M3 supports integration in M4, followed by scalability assessment in M5 and initial analysis in M6. M10 reviews the results and selects follow-up investigations for M11 before final validation in M12.

M7 will assess findings, validation evidence, limitations, and writing readiness to determine whether to proceed with conference submission. M8 proceeds if that review supports submission; M9 additionally depends on sufficiently validated findings and supporting analysis. The abstract and paper deadlines are December 18 and December 23,

2026, respectively [2]. M8 and M9 are conditional; the remaining capstone milestones apply regardless of the submission decision.

3.3 Further Exploration and Contingencies

Following the initial analysis in M6, the research review in M10 will select follow-up investigations for M11. Candidates include differences in skill revision activity across repository domains, instruction structure, and reuse patterns. Selection will depend on data availability, measurement validity, and remaining time.

Table 3: Contingencies and affected milestones

ID	Risk and trigger	Response and effect on the plan	Affected milestones
R1	**Incomplete data or repository history:** fields or revisions required for a calculation cannot be obtained.	Document missingness and restrict the affected analysis to eligible records. If insufficient evidence remains, report the measurement as unavailable and assess the effect on requirements and conclusions.	M2, M3, M6
R2	**Unsupported metric interpretation:** a metric fails its specified validation criteria.	Review preprocessing, assumptions, and the measurement method, then repeat validation. If it remains unsupported, report its limitations and exclude interpretations that the evidence does not justify.	M3, M6, M10, M12
R3	**Processing limits:** measured runtime or memory exceeds the specified acceptance limit (PF-01, PF-02).	Adjust batch sizes or data access and repeat performance tests. If full-dataset processing remains infeasible, propose documented sampling and explicitly review its effect on coverage and performance requirements.	M5, M6, M12
R4	**Schedule pressure:** unresolved defects or analysis delays threaten required validation or delivery.	Reassign work and defer optional investigations to protect core analysis and validation. Forgo MSR submission if necessary and continue toward capstone deadlines.	M6, M7, M8, M9, M11, M12

Changes affecting mandatory requirements will be reviewed with the supervisor and reflected in the requirements, tests, and schedule. Optional investigations may be deferred, but unsuccessful tests or unsupported findings will not be presented as completed validation.

4. Testing and Evaluation

Verification will check metric calculations against independently established reference results and assess data handling, pipeline integration, reproducibility, and performance. Validation will assess whether the measurements support their intended interpretation and the research questions. Each metric owner will document the test inputs, procedure, expected results, acceptance criteria, and associated requirement and prototype IDs.

4.1 Metric Verification and Validation

Table 4: Planned metric verification and validation

Test ID	Metric	Planned verification
T01	Size	*Inputs:* 20 skills chosen from the 13,000-skill sample to cover prose-only bodies, bodies with code blocks, bundles with both text and binary files, and at least one skill with *composition_truncated* = 1. Expected counts come from a short standalone script that tokenizes each tier's text on its own and counts lines and words, and the split between description, body, and bundle is checked by eye. *Procedure:* run pre-processing and the Size module on the 20 skills. *Acceptance:* every count matches the reference script, binary files are left out of the bundle, and every truncated skill is flagged. *Requirements:* FR-01 to FR-04, IF-02, PF-07.

Test ID	Metric	Planned verification
T02	Churn	*Inputs:* 20 skills chosen from the sample, made up of 10 skills with commit history, 5 edited skills whose repositories are cloned (at least one of them renamed), and one near-clone family of about 5 members. Expected values are worked out by hand from the commit fields and dates, the AI and bot flags are read from the commit messages, the tokens added and removed come from git's word diff of each version and copy pair, and the order of the copies is read from the dates. *Procedure:* run the Churn module on these skills, then have the team label 100 links from the full sample as adapted copy, template only, or unrelated. *Acceptance:* activity values and flags match exactly, token counts are within 2 tokens of git's diff, the history is followed through the rename, each copy in the family is linked to an earlier member, with the direction matching the dates where they differ, and the labelled links meet PF-04. *Requirements:* FR-05 to FR-08, IF-04, IF-05, PF-04, PF-08.
T03	Age	Compare calculated ages with expected values from known timestamps and a fixed reference date.
T04	Clone coverage	Compare detected clone groups and reported coverage with a controlled collection whose duplication relationships are known.
T05	Readability	Compare implemented scores with reference calculations; separately assess whether the scores support the intended readability interpretation.

4.2 Pipeline Verification

Table 5: Planned pipeline verification

Test ID	Procedure	Proposed acceptance criterion
T06 Data handling and integration	Process controlled records, including missing fields and duplicate content, through preprocessing and the integrated pipeline.	Every input record has the expected retained, excluded, or flagged outcome; retained identifiers and metric values remain consistent through output generation.

Test ID	Procedure	Proposed acceptance criterion
T07 Reproducibility	Repeat analysis using the same dataset snapshot, configuration, and software versions. Compare metric tables and values used in plots.	No differences in deterministic metric values after matching records by identifier; any permitted numerical tolerance is explicitly defined in the corresponding requirement.
T08 Performance	Measure elapsed runtime and peak memory for the workload and workstation specified in the performance requirements.	Runtime and peak memory satisfy the stated acceptance limits for that workload and environment.

Initial metric verification supports M3, integration testing supports M4, and performance testing supports M5. Output checks support the initial analysis in M6, while interpretation and limitations are reviewed in M10. Applicable tests will be repeated for final validation in M12.

4.3 Evaluation and Acceptance

Test records will identify the dataset or controlled inputs, configuration, expected and actual results, and pass/fail outcome. Failed checks will be investigated and affected tests repeated after corrections. Measurements that fail validation will not support interpretations beyond the evidence available; inconclusive research findings alone do not constitute a test failure.

5. Resource Requirements

5.1 Scheduling access to resources

Table 6: Resource access by milestone.

Milestone	Resource / source	Access date / lead time
M2	GitSkills 13,000-skill sample, Python, GitHub repository, VS Code/Jupyter, data-processing libraries	Immediate, publicly available and open source
M3–M6	Full GitSkills dataset, local storage, Python analysis environment, source-code comparison dataset	Required before full-scale analysis, sufficient storage required
M10–M11	Statistical, text-analysis, tokenization, and visualization libraries, processed metric files	Required during metric calculation, validation, and full-dataset analysis
M12	GitHub repository, processed results, plots, validation and reproducibility scripts	Continuous access through final validation and reporting

5.2 Materials and other resources

Table 7: Preliminary cost estimate (CAD).

Item / source	Qty.	Unit cost	Total
GitSkills dataset and official sample	1	$0	$0
Python, Git, VS Code/Jupyter, and open-source analysis libraries	1 set	$0	$0
GitHub repository and version-control access	1	$0	$0
Local data storage	1	$0	$0
Estimated total			**$0**

6. Conclusion

This project aims to examine size, churn, age, clone coverage, and readability within AI agent skills using the GitSkills dataset. The team plans to study how these skills are developed and maintained, and whether traditional software engineering metrics can be meaningfully applied to them. The final system will be a Python analysis pipeline that processes the dataset and calculates metrics based on the available fields.

Development will follow twelve milestones (M1–M12). M2 focuses on dataset preparation and verification of sample records. Next, M3 involves the initial metric calculations and checking them against reference examples. Once the methods have been established, the team will calculate the supported metrics across the dataset and generate preliminary tables and visualizations (M6). This is followed by M10, which involves reviewing results and documenting focused research questions. Based on these findings, the team will investigate follow-up questions and potential extensions during M11. Finally, the pipeline will undergo validation and reproducibility testing in M12. The pipeline will be designed to process the full dataset within 24 hours while using no more than 16 GB of memory. Testing will include unit tests, integration tests, and reproducibility checks to ensure the calculated metrics are accurate and consistent. Since the project relies on publicly available data and open-source tools, the estimated total cost is $0 CAD.

The expected outcome of the project is to identify and document meaningful metrics that can help developers and researchers better understand AI agent skills and how they evolve. The full scope of the project will depend on the results of preliminary analysis, as focused research questions will be established during M10 and investigated further during M11. Although this makes part of the project's scope open-ended, the team has established a clear development plan, assigned milestones, and identified the resources needed for success. With initial pilot analyses already completed, the team is prepared to move forward with implementation and validation.

7. References

[1] G. Destefanis, D. Graziotin, M. Vaccargiu, and M. Ortu, "GitSkills: A dataset of agent skills on GitHub," in *Proceedings of the 24th International Conference on Mining Software Repositories (MSR '27)*, To appear, Dublin, Ireland: IEEE, 2027. DOI: 10.48550/arXiv.2608.10906 arXiv: 2608.10906.

[2] *MSR 2027 Mining Challenge: Call for Mining Challenge Papers*, https://conf.researchr.org/track/msr-2027/msr-2027-mining-challenge, Accessed: Oct. 7, 2026, International Conference on Mining Software Repositories.

[3] R. P. L. Buse and W. R. Weimer, "Learning a metric for code readability," *IEEE Transactions on Software Engineering*, vol. 36, no. 4, pp. 546–558, 2010. DOI: 10.1109/TSE.2009.70

[4] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 2nd ed. Thousand Oaks, CA: Sage, 2004.

[5] R. Flesch, "A new readability yardstick," *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948. DOI: 10.1037/h0057532

[6] J. P. Kincaid, J. Fishburne Robert P., R. L. Rogers, and B. S. Chissom, "Derivation of new readability formulas (automated readability index, fog count and Flesch reading ease formula) for Navy enlisted personnel," Naval Technical Training Command, Millington, TN, Research Branch Report 8-75, 1975.

[7] D. Posnett, A. Hindle, and P. Devanbu, "A simpler model of software readability," in *Proceedings of the 8th Working Conference on Mining Software Repositories (MSR '11)*, ACM, 2011, pp. 73–82. DOI: 10.1145/1985441.1985454

[8] textstat contributors, *Textstat: Python package to calculate readability statistics of a text*, version 0.7.13. Accessed: Oct. 7, 2026. [Online]. Available: https://github.com/textstat/textstat

A. Supporting Schedule and Technical Material: N/A

B. Readability Pilot

We ran the readability method on the official 13,000-skill GitSkills sample to check that it is feasible and to choose the preprocessing. Of the 13,000 distinct skills, 237 were not named exactly `SKILL.md`, 1,566 had invalid front matter, 555 were too short (a body under 300 characters or fewer than 30 words of prose), and 2,328 were not in English. This left 8,314 skills from 5,567 repositories. Figure 3a shows why the Markdown must be converted into sentences before scoring: on raw text, each bullet list counts as one long sentence, and the median grade rises from 8.6 to 12.2. Figure 3b shows that skill sentences are short (a median of 9 words), but formulas that weight word length, such as Coleman–Liau, place the same text near grade 13, which reflects technical vocabulary. Descriptions are usually a single long sentence, and they read about four grades harder than the bodies. The pilot code uses textstat [8]; whole-sample scoring took about 6 ms per skill on one laptop core, or roughly 3 h for the full 1.88 million distinct skills.

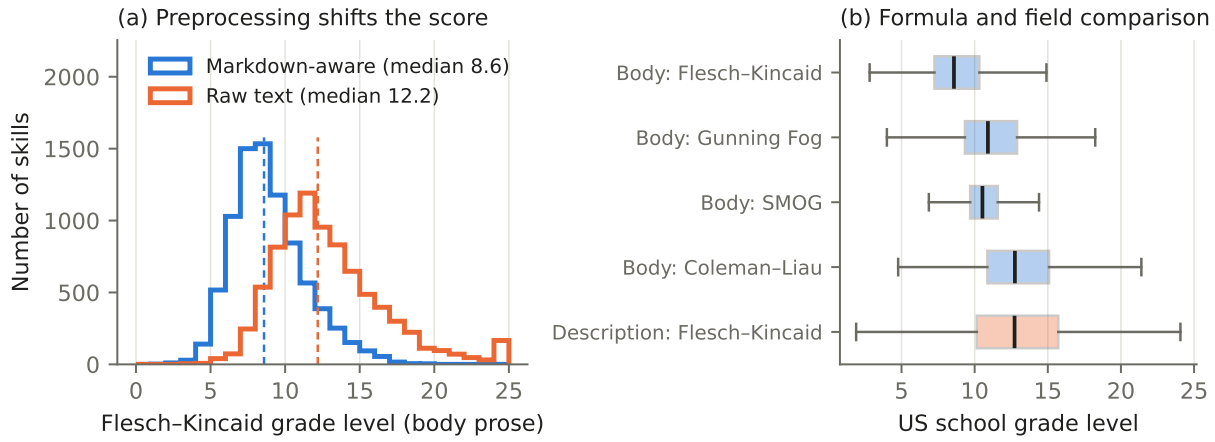


Figure 3: Readability pilot on 8,314 English skills from the 13,000-skill GitSkills sample. (a) Flesch–Kincaid grade of the body prose, with the Markdown converted to sentences and as raw text. (b) Body grade under four formulas, and Flesch–Kincaid grade of the front-matter description. Boxes show the interquartile range, and whiskers extend to 1.5 times the IQR.